GB3 RISC-V Processor Project Plan

Group 4 | Gijeong Lee, George Wormald, Lonnie McKiernan | May 28, 2026

Project Objective

Our team plans to improve loop-intensive integer computation in areas like embedded control algorithms, digital communications decoding, and signal processing, where the same instruction and data regions are repeatedly accessed (the final benchmark workload will be selected from these categories following collective research in Session 5.) The current bottleneck is instruction fetch latency, which we calculated to waste approximately 88% of execution cycles on redundant flash re-fetches. We will address this through three progressive phases: first implementing a direct-mapped instruction cache, then extending to a data cache, with a final upgrade to a 4-word-line 2-way set-associative cache as an extension goal.

Baseline Characterisation

Preliminary measurements from the Session 3 standard firmware benchmark (Command 9) recorded a CPI of 8.54 against a 12 MHz clock, giving a measured execution time of $253{,}741 \times 8.54 \times 83.3\,\text{ns} \approx 180\,\text{ms}$. With an ideal CPI of approximately 1 for a simple in-order processor, roughly 88% of all execution cycles are stalled waiting for SPI flash, making instruction caching the highest-leverage modification available. These measurements were taken under `firmware.c`; final baseline CPI and execution time will be re-established under the project benchmark workload on unmodified hardware at the start of Session 5, verified against Picoscope LED toggle period measurement.

Target Metrics

Performance (CPI reduction) is the primary target; area and power are monitored as secondary constraints.

Metric	Baseline	Target	Justification
Performance (CPI)	∼8.54	< 2.5	Assuming >90% hit rate for loop-dominated code, avg. CPI $\approx 0.9 \times 1 + 0.1 \times 8 = 1.7$ for Phase 1; target of <2.5 is conservative; Phases 2 and 3 expected to reduce CPI further
Area (LC util.)	79%	$< 85\%$	Cache uses EBR block RAM not logic cells; 85% cap leaves synthesis timing margin above the 79% baseline
Power	∼600 mW	≤570 mW	Fewer flash accesses per loop expected to reduce dynamic switching; 5–10% reduction estimated; exact magnitude to be measured

CPI was measured under `firmware.c` benchmark in Session 3 and final baseline will be re-established using the project benchmark workload in Session 5. Power was measured at ∼600 mW under `blink.c` in Session 3 and will similarly be re-measured in Session 5.

Bottleneck Analysis

- For a loop executing N iterations over K instructions, the same K instructions are fetched from flash $N \times K$ times at ∼8 cycles each. After the first iteration, all $(N-1) \times K$ remaining fetches will result in cache hits, allowing the CPU to access instructions significantly faster.
- Constant lookup tables and pre-computed coefficient arrays stored in flash (such as filter coefficients, state matrices, or parity tables) are read at the same addresses on every inner loop iteration. Such data is never modified at runtime, so future reads can be served from the data cache without accessing flash.
- A simple prefetch buffer only serves sequential access; a backward branch at the end of a loop always causes a miss and re-fetches from flash. A tagged direct-mapped cache handles loop-back branches correctly after cold start, making it strictly more effective for loop-dominated workloads.
- SPRAM data reads complete in ∼2 cycles and are not a bottleneck; the inefficiency is exclusively in instruction fetch and read-only constant data residing in flash.

Proposed Modifications

Supporting parameter changes: `ENABLE_FAST_MUL=1` (integer multiply: $30 \rightarrow 3$ cycles) and `ENABLE_COMPRESSED=0` (per project specification: `-march=rv32im`).

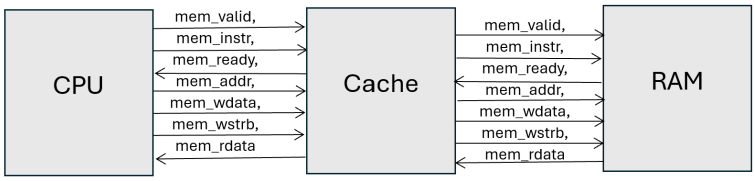

Figure 1: CPU–cache–flash block diagram showing PicoRV32 native memory interface signals intercepted by the cache module. The right-hand block represents SPI flash accessed via `spimemio`.

Phase 1: Direct-Mapped Instruction Cache

Caches are smaller but faster to access than main memory, exploiting temporal locality to speed up instruction throughput. The simplest form is a direct-mapped cache, which assigns each memory address to exactly one cache line via a modulo mapping, avoiding the need for a replacement algorithm and minimising implementation complexity. An initial design of 16 sets with 1-word (32-bit) lines will be used, expandable after performance analysis. Cache is applied to instructions only in this phase.

On a tag match with a valid entry (hit), the instruction is served in 1 cycle with no flash access. On a miss, a single word is fetched from flash, the cache entry is updated, and the instruction is returned to the CPU. The key integration challenge is inserting `icache.v` between the `mem_valid/mem_ready` handshake of `picorv32` and `spimemio`, such that `mem_ready` is asserted by the cache on a hit rather than by `spimemio`.

For loop-dominated workloads where the same K instructions execute N times, the cache reduces instruction fetch cost from $N \times K \times 8$ cycles to approximately $K \times 8 + (N-1) \times K \times 1$ cycles after cold start — a reduction proportional to the loop iteration count.

Files changed: new `icache.v`; modified `picosoc.v`.

Phase 2: Data Cache

Phase 2 adds a separate data cache module (`dcache.v`) handling `mem_instr=0` data read transactions. Together with `icache.v`, this forms a Harvard-style split cache architecture, allowing the CPU to simultaneously fetch an instruction and load data without bus contention. The data cache targets constant lookup tables and arrays stored in flash (`.rodata` section). Since these are never written at runtime, no write-back logic is required, keeping the implementation as simple as Phase 1. Phase 1 alone constitutes a complete and independently measurable contribution; Phase 2 is pursued only if Phase 1 is complete by end of Session 5.

Files changed: new `dcache.v`; modified `picosoc.v`.

Phase 3: 4-Word Line, 2-Way Set-Associative Cache

Phase 3 upgrades the instruction cache with two simultaneous improvements. First, cache lines are expanded from 1 word to 4 words, exploiting spatial locality: on a miss, 4 consecutive instructions are fetched and cached, so sequential code following a miss is already available without further flash accesses. Second, the cache is upgraded from direct-mapped to 2-way set-associative, eliminating conflict misses where two addresses mapping to the same line evict each other. Each set holds 2 ways; the cache reads both simultaneously and a multiplexer selects the matching tag. A 1-bit LRU state per set determines eviction on a miss. Phase 3 is a stretch goal pursued only if Phase 2 is complete and verified.

Files changed: modified `icache.v`.

Division of Labour

Prior to implementation, all members will collectively research existing RISC-V cache implementations, study relevant benchmark workloads, and establish a shared understanding of the `picorv32/picosoc` memory interface. Once implementation begins, roles are partitioned by file domain to prevent merge conflicts. George's benchmark must produce a verified correct output before Lonnie can begin performance measurement; this dependency resolves early in Session 5.

Member	Role	Session 5 (29 May)	Session 6 (2 June)
Gijeong	*Hardware Architect*	Design `icache.v`; integrate into `picosoc.v`; resolve `mem_valid/mem_ready` handshake; synthesis and timing verification	Hardware test of `icache.v` on board; implement `dcache.v` (Phase 2) if Phase 1 complete; begin 2-way associative upgrade (Phase 3) if time permits
George	*Firmware & Verification*	Write `run_workload()` benchmark; verify output correctness against reference; write `icache_tb.v`; run `make icebsim`; confirm hit/miss behaviour in simulation	Extend testbench to `dcache_tb.v` and Phase 3 variant; validate combined simulation; confirm `run_workload()` output unchanged; secret benchmark submission (4 June)
Lonnie	*Measurement & Analysis*	Baseline PPA measurements (CPI, area, power); $N \times \text{CPI} \times T$ calculation; Picoscope LED period capture; document baseline results	Post-modification PPA for each completed phase; before/after comparison table; Picoscope captures; compile and document final results

Development Timeline

1. Establish baseline PPA measurements before any hardware modifications.
2. `icache.v` stage 1: Design and simulate to only pass wires through to flash — confirm the CPU works as before.
3. `icache.v` stage 2: Add storage arrays for tag bits, valid bits and data; always cache miss and verify data and tags are written correctly.
4. `icache.v` stage 3: Add hit logic; read from cache on a hit.
5. Write and verify `run_workload()` benchmark — correct output confirmed against reference implementation.
6. Hardware test of instruction cache on iCEBreaker; measure Phase 1 PPA.
7. Implement Phase 2 data cache if Phase 1 complete.
8. Implement Phase 3 4-word line, 2-way set-associative cache if Phase 2 complete.
9. Run final PPA measurements for all completed phases.
10. Submit secret benchmark (deadline: 4 June).